Wow, three errors… a notice and two warnings. Nothing fatal, but our code doesn't do what we hoped for. This is because static properties cannot be accessed with the same syntax used to access non-static (the object operator `->`). We have to access them using a different operator, and the syntax for doing so is given in the Notice error in the output:

```
print QuoteBot::$quoteStash[1];
```

```
Output:
I'll show you the life of the mind!
```

The double-colon operator between `QuoteBot` and `$quoteStash` is called the *scope resolution operator*, which isn't very intuitive (this has nothing to do with global vs. function lexical scope). A better name might be something like context resolution operator, or class context operator, because its purpose is to express that the target property or method does not exist in object context. (Still, it's more intuitive than its internal name of `PAAMAYIM NEKUDOTAYIM`, which is Hebrew for "double colon.")

As you can see, we don't need to instantiate a `QuoteBot` object to access a `static` member, but (unlike access via an object) we do need to use the dollar symbol with the property name.

Accessing static methods on objects works in the same way. Let's add one to `QuoteBot`:

```
    public static function insight(): string
    {
        return "I've always found that writing comes from a great inner
        pain." . PHP_EOL;
    }
```

The syntax for calling the code is as you'd expect:

```
print QuoteBot::insight();
```

```
Output:
I've always found that writing comes from a great inner pain.
```

Although we haven't used an object instance, static members are also accessible from objects in the following way:

```
print $quoteBot::insight(); // not $quoteBot->insight()
print $quoteBot::$quoteStash[0]; // not $quoteBot->$quoteStash
```